MYSQL CUSTOMER CHURN ANALYSIS

E-Commerce Customer Churn Dataset

Module End Assignment 2 - MySQL

Detail	Information
Student Name	ANCY.V
Course	AI DRIVEN DATA ANALYTICS
Submission Date	12/10/2026
Tool Used	MySQL Workbench 8.0
Database / Table	ecomm / customer_churn

1. Introduction

Customer churn means customers stop buying from a business. In e-commerce, keeping an existing customer usually costs far less than winning a new one, so understanding why customers leave is essential for profit and growth. This assignment uses MySQL to study an e-commerce customer dataset and find the patterns behind churn.

The work has three stages. First, the raw data is cleaned: missing values are filled, unrealistic outliers are removed and inconsistent labels are standardised. Second, the table is transformed by renaming columns, creating readable status columns and dropping the old numeric ones. Third, 18 business questions are answered with SQL queries using filtering, grouping, aggregate functions, CASE statements, subqueries and a join.

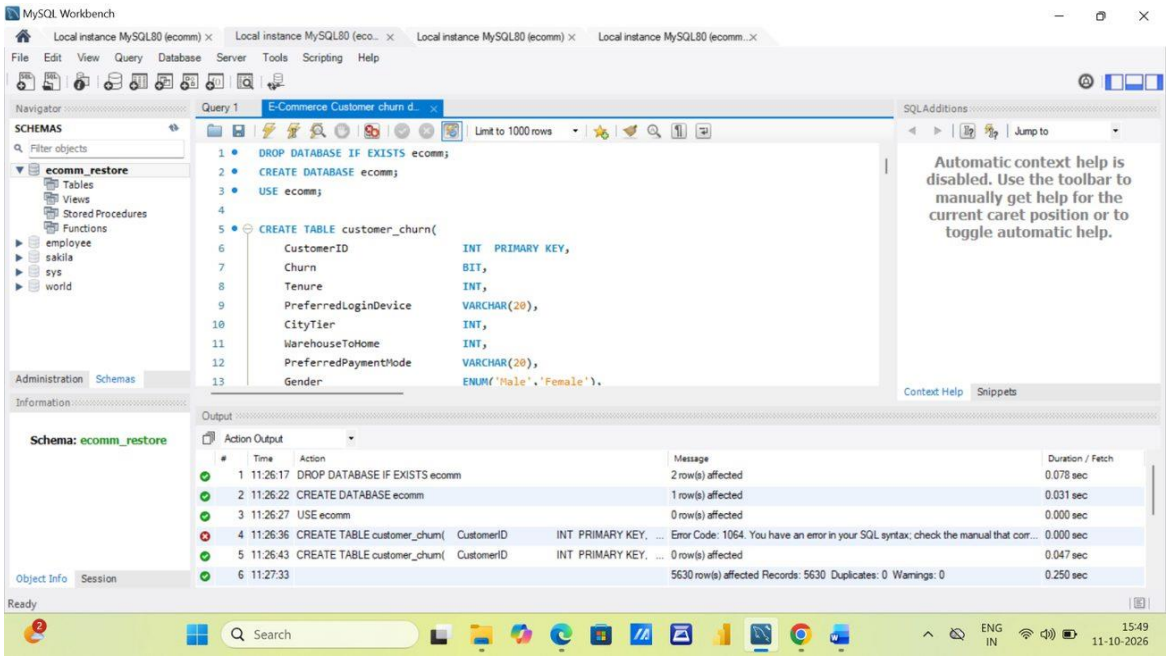
2. Objectives

- Clean the data by handling missing values, outliers and inconsistent category names.
- Transform the data with clearer column names and readable ChurnStatus and ComplaintReceived columns.
- Answer the business questions about churn, tenure, complaints, payment modes, order categories, cashback, coupons and delivery distance.
- Summarise the findings and recommend ways to reduce customer churn.

3. Dataset Overview

Item	Details
Database	ecomm
Main table	customer_churn
Rows (raw data)	5,630 customers
Columns (20)	CustomerID, Churn, Tenure, PreferredLoginDevice, CityTier, WarehouseToHome, PreferredPaymentMode, Gender, HourSpendOnApp, NumberOfDeviceRegistered, PreferedOrderCat, SatisfactionScore, MaritalStatus, NumberOfAddress, Complain, OrderAmountHikeFromlastYear, CouponUsed, OrderCount, DaySinceLastOrder, CashbackAmount

Each row represents one customer. Churn is 1 when the customer has left and 0 when the customer is still active. The provided SQL file was run in MySQL Workbench to create the ecomm database and the customer_churn table and to insert all 5,630 records, as the screenshot below shows (Records: 5630, Warnings: 0).



4. Data Cleaning

Data cleaning makes the data accurate and consistent before analysis. Wrong or missing values would give misleading answers, so each issue is handled in the order given in the assignment.

4.1 Checking missing values

Explanation: Before filling anything, the number of NULL values in each column is counted. SUM(column IS NULL) counts the rows where the column is empty. MySQL Workbench also has safe update mode switched on, which blocks UPDATE and DELETE statements that do not use a key column in the WHERE clause, so it is switched off with SET SQL_SAFE_UPDATES = 0.

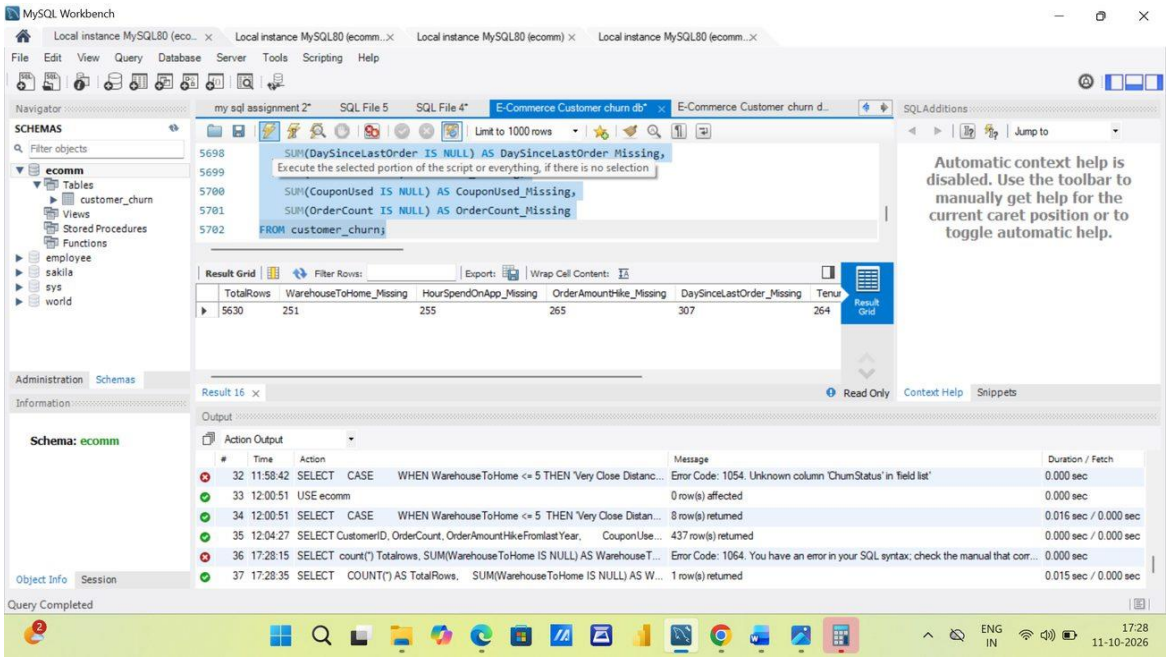
```
SET SQL_SAFE_UPDATES = 0;

SELECT COUNT(*) AS TotalRows,
       SUM(WarehouseToHome IS NULL)           AS WarehouseToHome_Missing,
       SUM(HourSpendOnApp IS NULL)            AS HourSpendOnApp_Missing,
       SUM(OrderAmountHikeFromlastYear IS NULL) AS OrderAmountHike_Missing,
       SUM(DaySinceLastOrder IS NULL)         AS DaySinceLastOrder_Missing,
       SUM(Tenure IS NULL)                    AS Tenure_Missing,
       SUM(CouponUsed IS NULL)                AS CouponUsed_Missing,
       SUM(OrderCount IS NULL)                AS OrderCount_Missing
FROM customer_churn;
```

Result

Column	Missing values
WarehouseToHome	251
HourSpendOnApp	255
OrderAmountHikeFromlastYear	265
DaySinceLastOrder	307
Tenure	264
CouponUsed	256

Column	Missing values
OrderCount	258



4.2 Imputing the mean (continuous columns)

Explanation: WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear and DaySinceLastOrder are continuous measurements, so missing values are replaced with the column average, rounded to the nearest whole number. The average is calculated first, and then each NULL is replaced with that value. Every UPDATE has WHERE column IS NULL, so only the missing cells change and existing data is untouched.

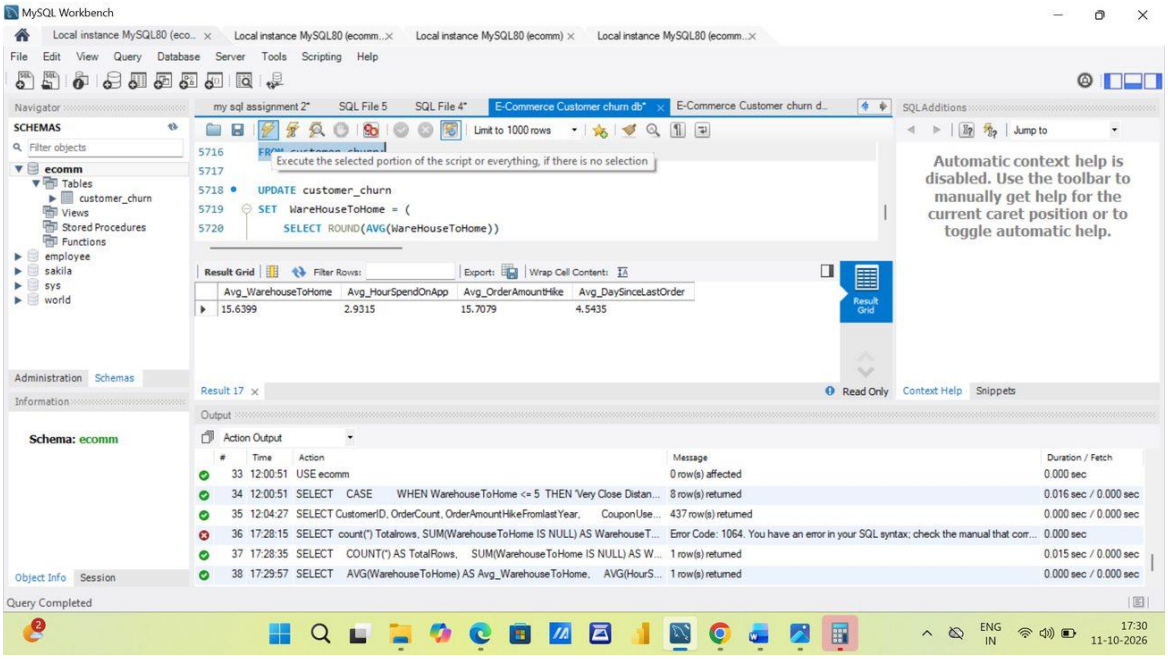
```
-- Step 1: find the averages
SELECT AVG(WarehouseToHome)           AS Avg_WarehouseToHome,
       AVG(HourSpendOnApp)            AS Avg_HourSpendOnApp,
       AVG(OrderAmountHikeFromlastYear) AS Avg_OrderAmountHike,
       AVG(DaySinceLastOrder)         AS Avg_DaySinceLastOrder
FROM customer_churn;

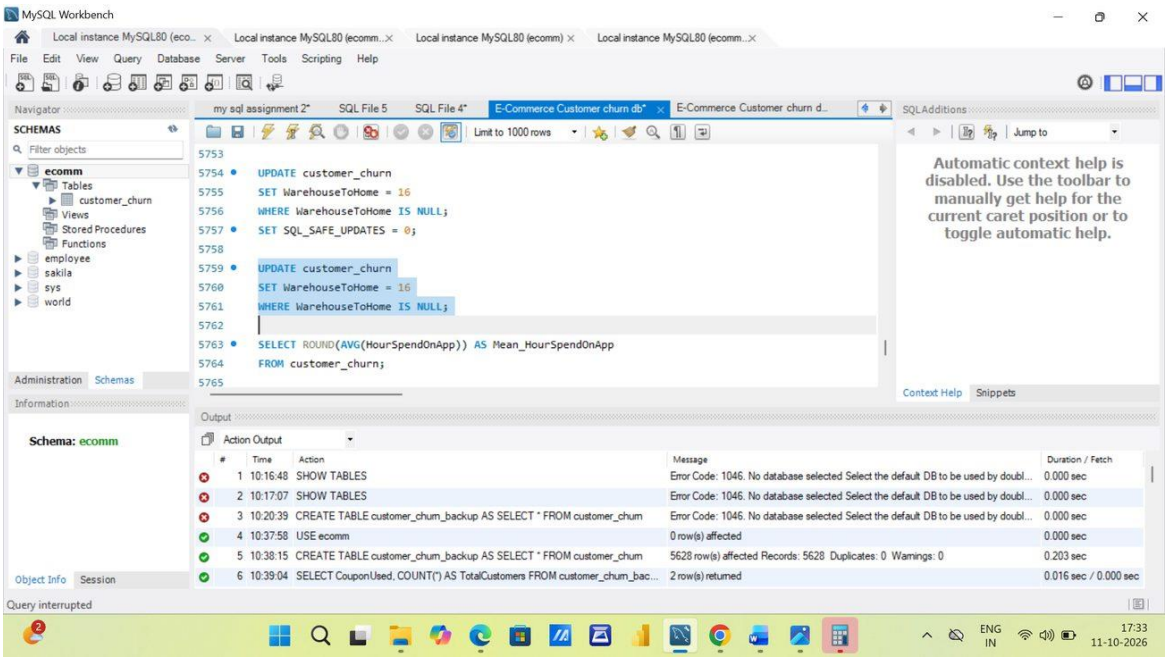
-- Step 2: fill the missing values with the rounded averages
UPDATE customer_churn SET WarehouseToHome = 16 WHERE WarehouseToHome IS NULL;
UPDATE customer_churn SET HourSpendOnApp = 3 WHERE HourSpendOnApp IS NULL;
UPDATE customer_churn SET OrderAmountHikeFromlastYear = 16
WHERE OrderAmountHikeFromlastYear IS NULL;
UPDATE customer_churn SET DaySinceLastOrder = 5 WHERE DaySinceLastOrder IS NULL;
```

Result

Column	Average	Rounded value used
WarehouseToHome	15.6399	16
HourSpendOnApp	2.9315	3

Column	Average	Rounded value used
OrderAmountHikeFromlastYear	15.7079	16
DaySinceLastOrder	4.5435	5





4.3 Imputing the mode (count columns)

Explanation: Tenure, CouponUsed and OrderCount are whole-number counts, so the most common value (the mode) is a better fill than an average. The mode is found by grouping the values, sorting by frequency and keeping the top row with LIMIT 1. Each NULL is then replaced with that mode. The WHERE ... IS NULL condition is essential here: without it, every row would be overwritten with the mode and the real data would be lost.

```
SELECT Tenure, COUNT(*) AS Frequency FROM customer_churn
WHERE Tenure IS NOT NULL GROUP BY Tenure ORDER BY Frequency DESC LIMIT 1;

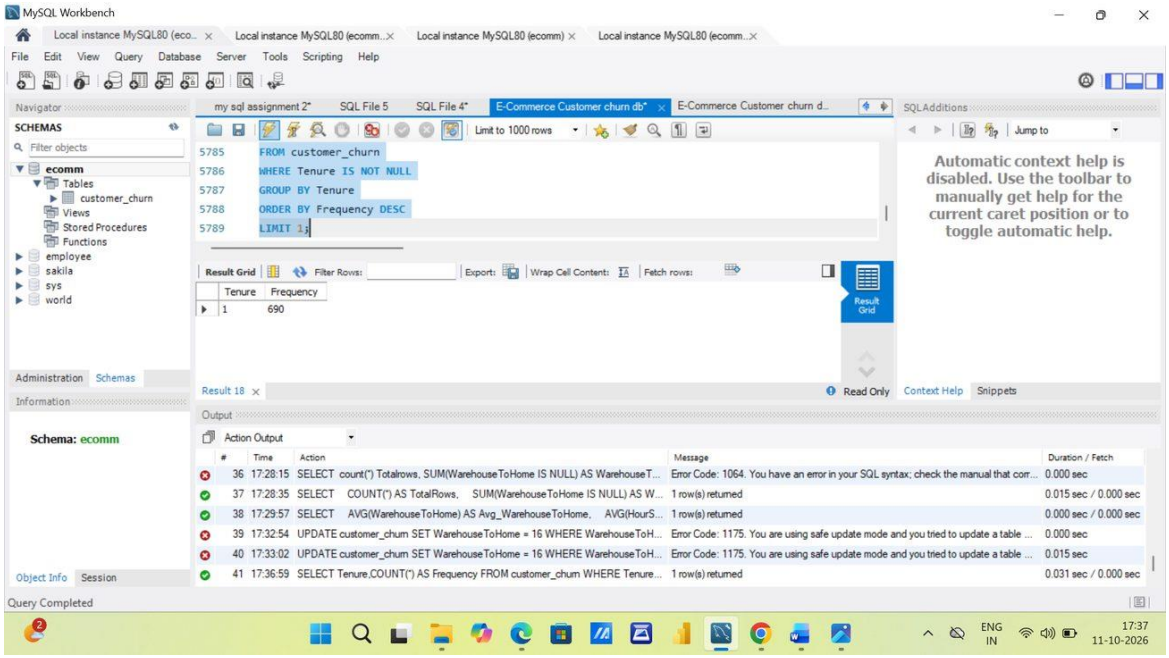
SELECT CouponUsed, COUNT(*) AS Frequency FROM customer_churn
WHERE CouponUsed IS NOT NULL GROUP BY CouponUsed ORDER BY Frequency DESC LIMIT 1;

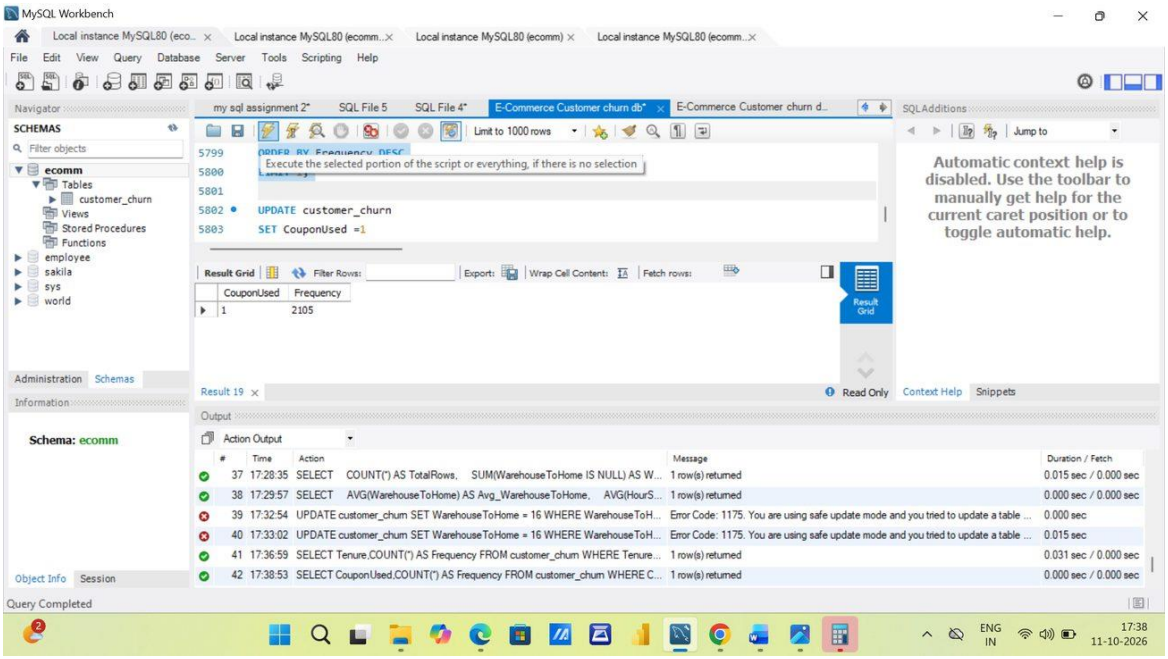
SELECT OrderCount, COUNT(*) AS Frequency FROM customer_churn
WHERE OrderCount IS NOT NULL GROUP BY OrderCount ORDER BY Frequency DESC LIMIT 1;


UPDATE customer_churn SET Tenure = 1 WHERE Tenure IS NULL;
UPDATE customer_churn SET CouponUsed = 1 WHERE CouponUsed IS NULL;
UPDATE customer_churn SET OrderCount = 2 WHERE OrderCount IS NULL;
```

Result

Column	Mode	Frequency
Tenure	1	690
CouponUsed	1	2,105
OrderCount	2	2,025





4.4 Handling outliers

Explanation: Almost all warehouse-to-home distances are below 40 km, so values above 100 km are unrealistic and are treated as data errors. The rows are counted first, then deleted, and the count is checked again to confirm they are gone.

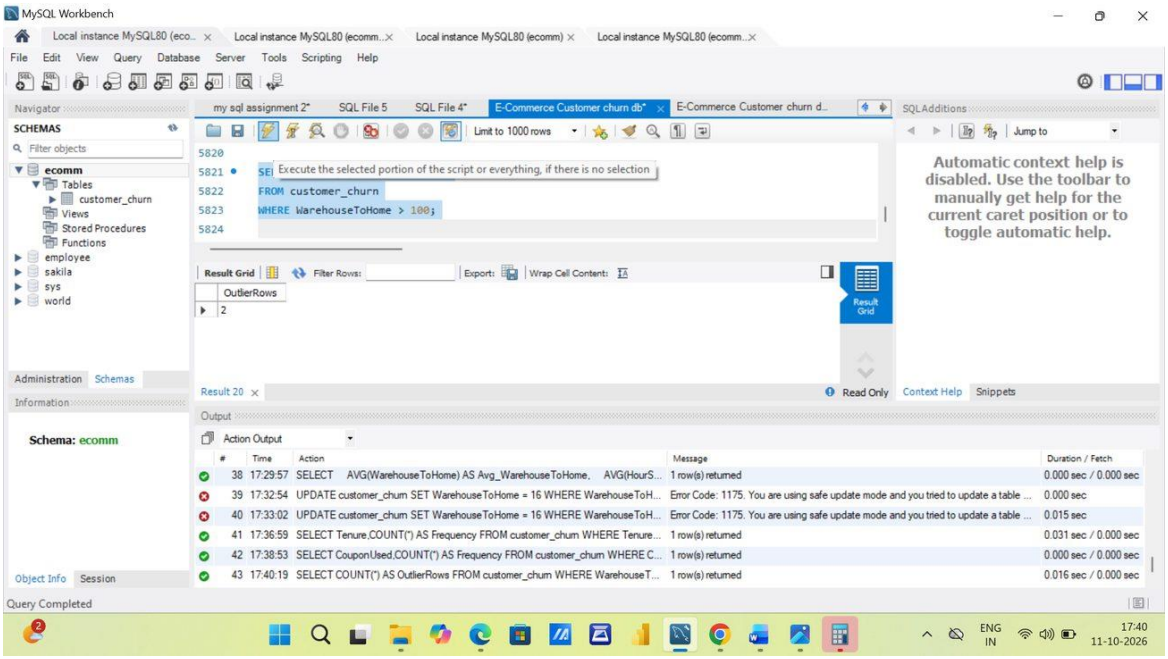
```
SELECT COUNT(*) AS OutlierRows FROM customer_churn WHERE WarehouseToHome > 100;

DELETE FROM customer_churn WHERE WarehouseToHome > 100;

SELECT COUNT(*) AS RemainingOutliers FROM customer_churn WHERE WarehouseToHome > 100;
SELECT COUNT(*) AS TotalRowsAfterCleaning FROM customer_churn;
```

Result

2 outlier rows were found (distance = 127 km). After the delete, 0 outliers remain and the table has 5,628 rows (5,630 - 2).



4.5 Fixing inconsistent labels

Explanation: The same value was written in different ways: "Phone" and "Mobile Phone" for the login device, "Mobile" and "Mobile Phone" for the order category, and "COD" / "Cash on Delivery" and "CC" / "Credit Card" for the payment mode. If left as they are, group-by queries would split one category into two. Exact matches (=) are used so values that are already correct are not changed again.

```
UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone'
WHERE PreferredLoginDevice = 'Phone';

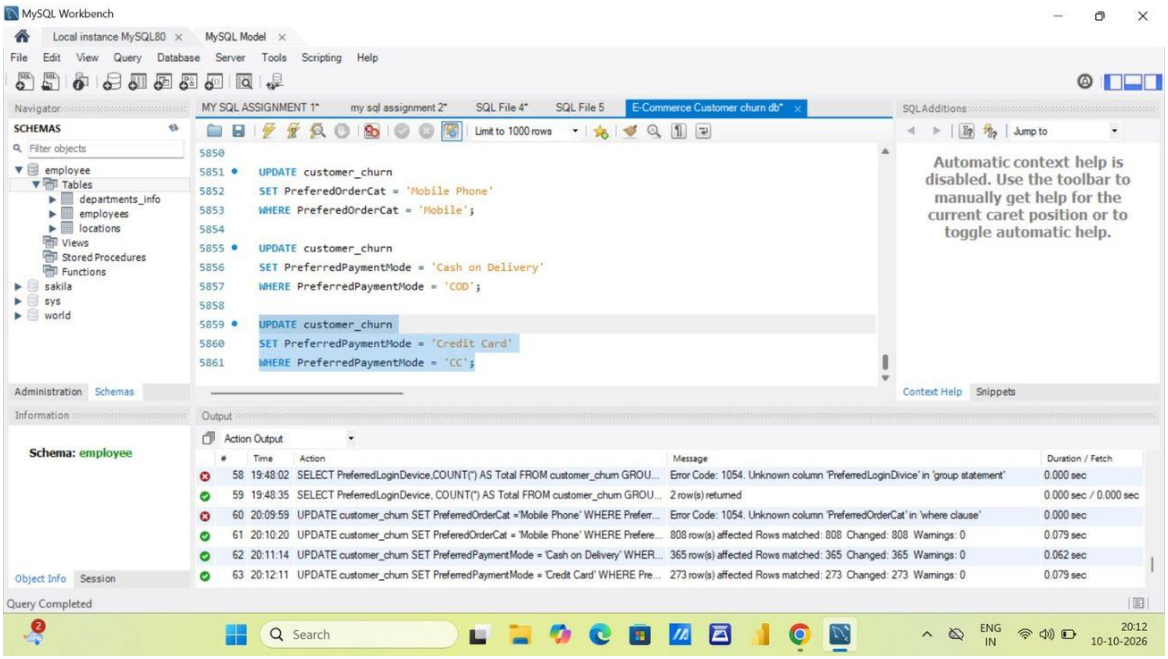
UPDATE customer_churn SET PreferedOrderCat = 'Mobile Phone'
WHERE PreferedOrderCat = 'Mobile';

UPDATE customer_churn SET PreferredPaymentMode = 'Cash on Delivery'
WHERE PreferredPaymentMode = 'COD';

UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card'
WHERE PreferredPaymentMode = 'CC';
```

Result

The updates changed 808 rows (Mobile to Mobile Phone), 365 rows (COD to Cash on Delivery) and 273 rows (CC to Credit Card). After standardising, PreferredLoginDevice has 2 values, PreferredPaymentMode has 5 values and the order category has 5 values.



5. Data Transformation

5.1 Renaming columns

Explanation: PreferedOrderCat contains a spelling mistake, and HourSpendOnApp reads better as HoursSpentOnApp. ALTER TABLE ... RENAME COLUMN changes the names without touching the data. From this point on, all queries use the new names.

```
ALTER TABLE customer_churn RENAME COLUMN PreferedOrderCat TO PreferredOrderCat;
ALTER TABLE customer_churn RENAME COLUMN HourSpendOnApp TO HoursSpentOnApp;

DESCRIBE customer_churn;
```

Result

The table structure now shows PreferredOrderCat and HoursSpentOnApp.

5.2 Creating the ComplaintReceived and ChurnStatus columns

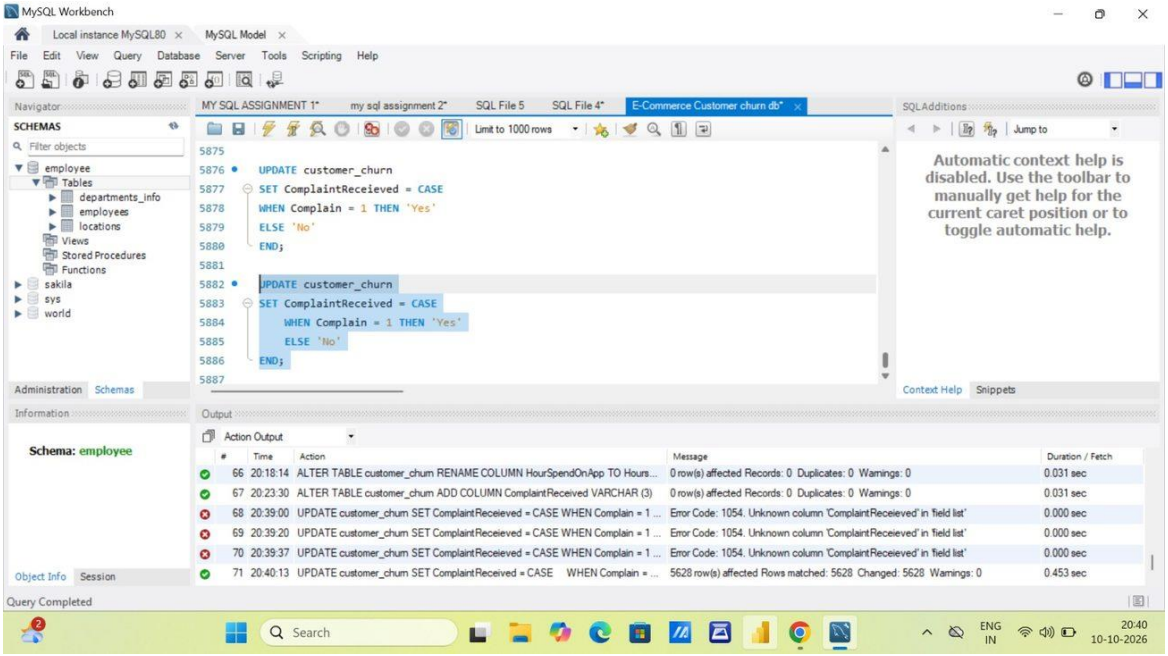
Explanation: Churn and Complain are stored as 1 and 0, which are hard to read in reports. Two new text columns are added with ALTER TABLE ... ADD COLUMN and filled with CASE statements: ComplaintReceived is "Yes" when Complain = 1 and "No" otherwise, and ChurnStatus is "Churned" when Churn = 1 and "Active" otherwise.

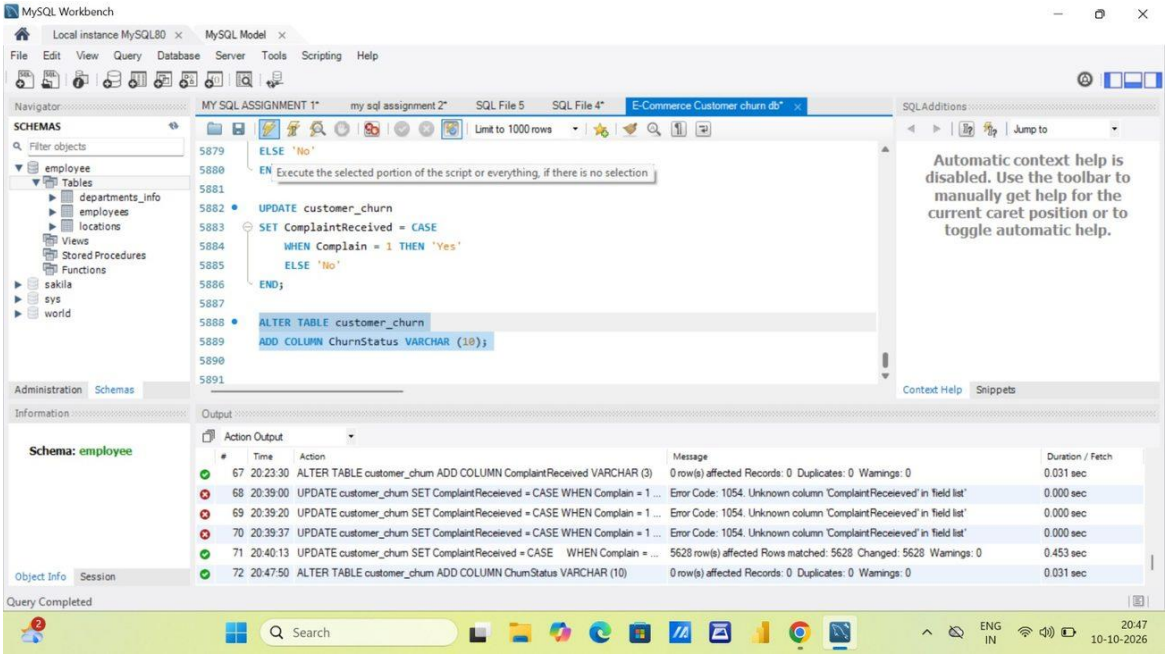
```
ALTER TABLE customer_churn ADD COLUMN ComplaintReceived VARCHAR(3);
UPDATE customer_churn
SET ComplaintReceived = CASE WHEN Complain = 1 THEN 'Yes' ELSE 'No' END;

ALTER TABLE customer_churn ADD COLUMN ChurnStatus VARCHAR(10);
UPDATE customer_churn
SET ChurnStatus = CASE WHEN Churn = 1 THEN 'Churned' ELSE 'Active' END;
```

Result

Both UPDATE statements matched and changed all 5,628 rows.





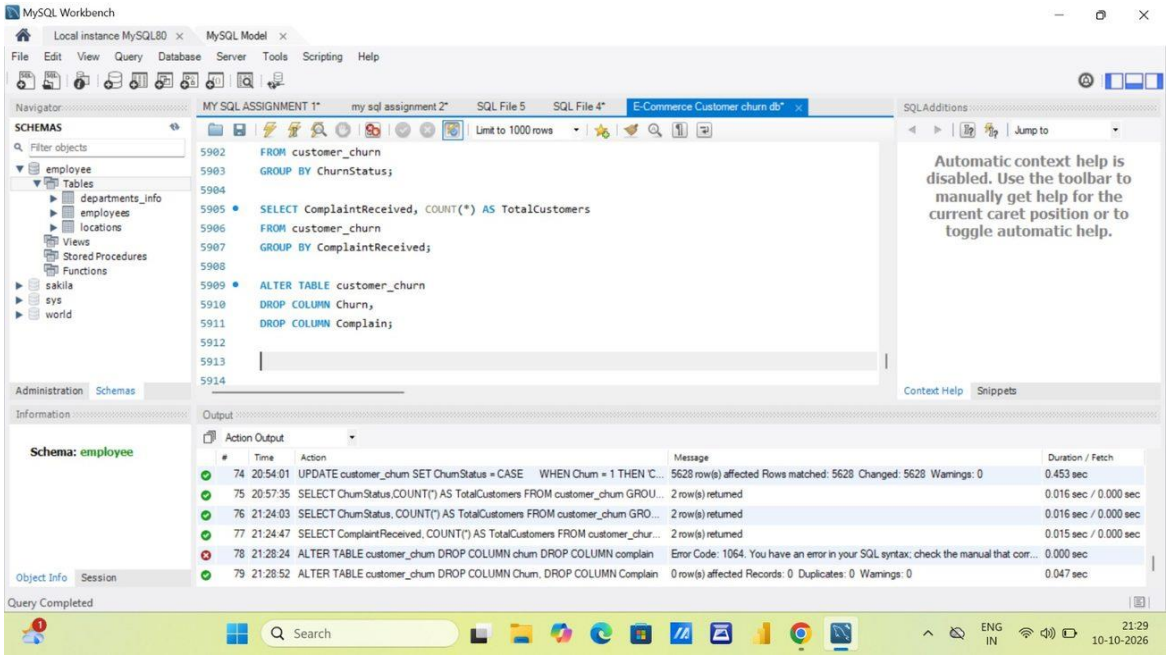
5.3 Dropping the original columns

Explanation: Once the readable columns exist, the original numeric Churn and Complain columns are redundant, so they are removed with ALTER TABLE ... DROP COLUMN.

```
ALTER TABLE customer_churn
DROP COLUMN Churn,
DROP COLUMN Complain;
```

Result

The statement ran successfully. The cleaned table now has 5,628 rows and 20 columns, including ChurnStatus, ComplaintReceived, PreferredOrderCat and HoursSpentOnApp. All analysis below uses this cleaned table.



6. Data Exploration and Analysis

Every query in this section runs on the cleaned customer_churn table.

Task 1 - Count of churned and active customers

GROUP BY ChurnStatus splits the customers into two groups, and COUNT(*) counts the rows in each.

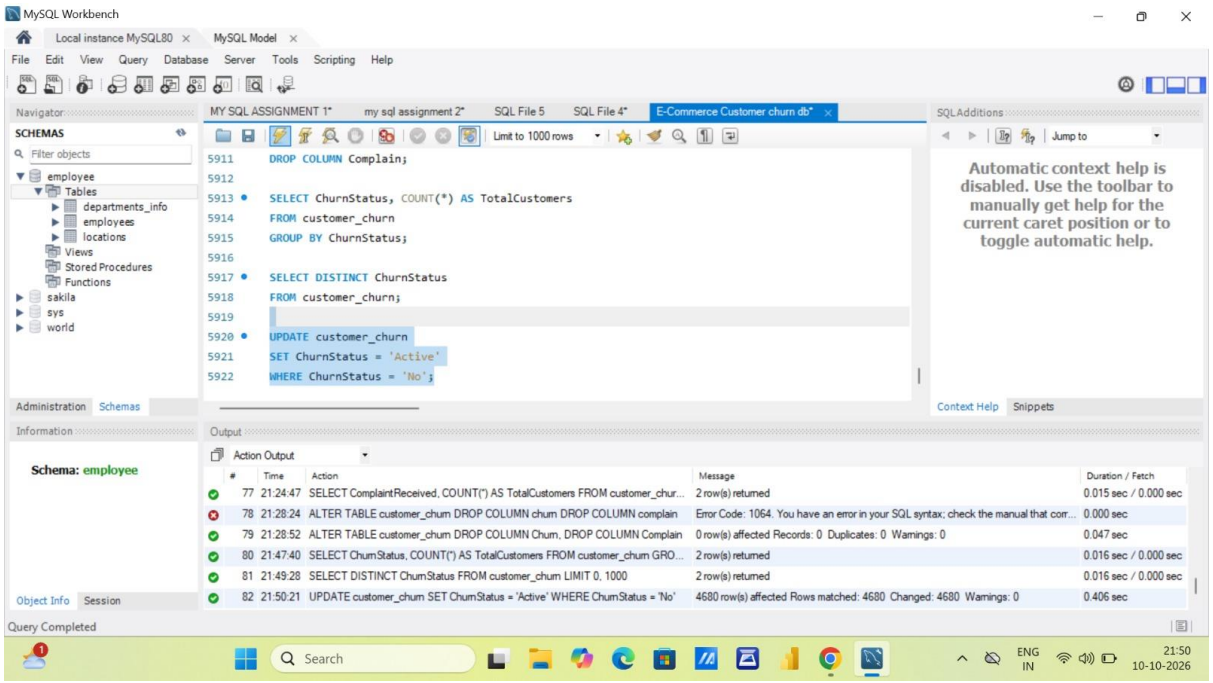
SQL query

```
SELECT ChurnStatus, COUNT(*) AS CustomerCount
FROM customer_churn
GROUP BY ChurnStatus;
```

Result

ChurnStatus	CustomerCount
Active	4680
Churned	948

Of 5,628 customers, 948 have churned, which is an overall churn rate of about 16.84%.



Task 2 - Average tenure and total cashback of churned customers

The WHERE clause keeps only churned customers. AVG gives the average tenure and SUM adds up the cashback amounts.

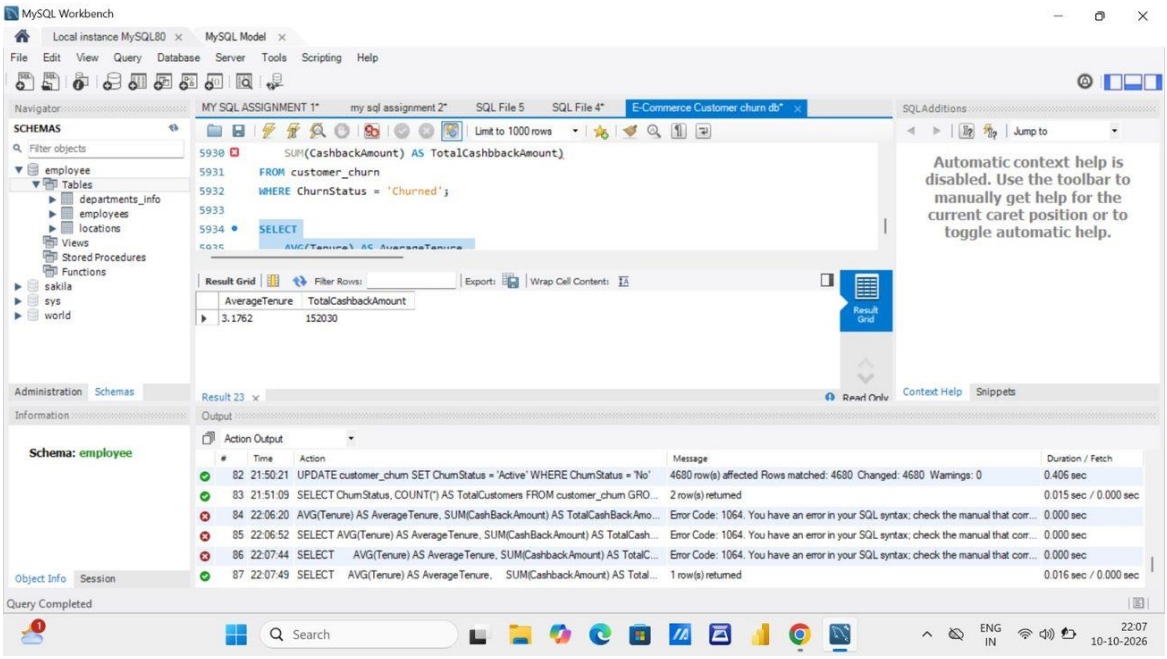
SQL query

```
SELECT AVG(Tenure) AS AverageTenure,
       SUM(CashbackAmount) AS TotalCashbackAmount
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Result

AverageTenure	TotalCashbackAmount
3.1762	152030

Churned customers stay only about 3 months on average, so most customers leave early.



Task 3 - Percentage of churned customers who complained

Among churned customers, the number with ComplaintReceived = Yes is divided by the total number of churned customers and multiplied by 100.

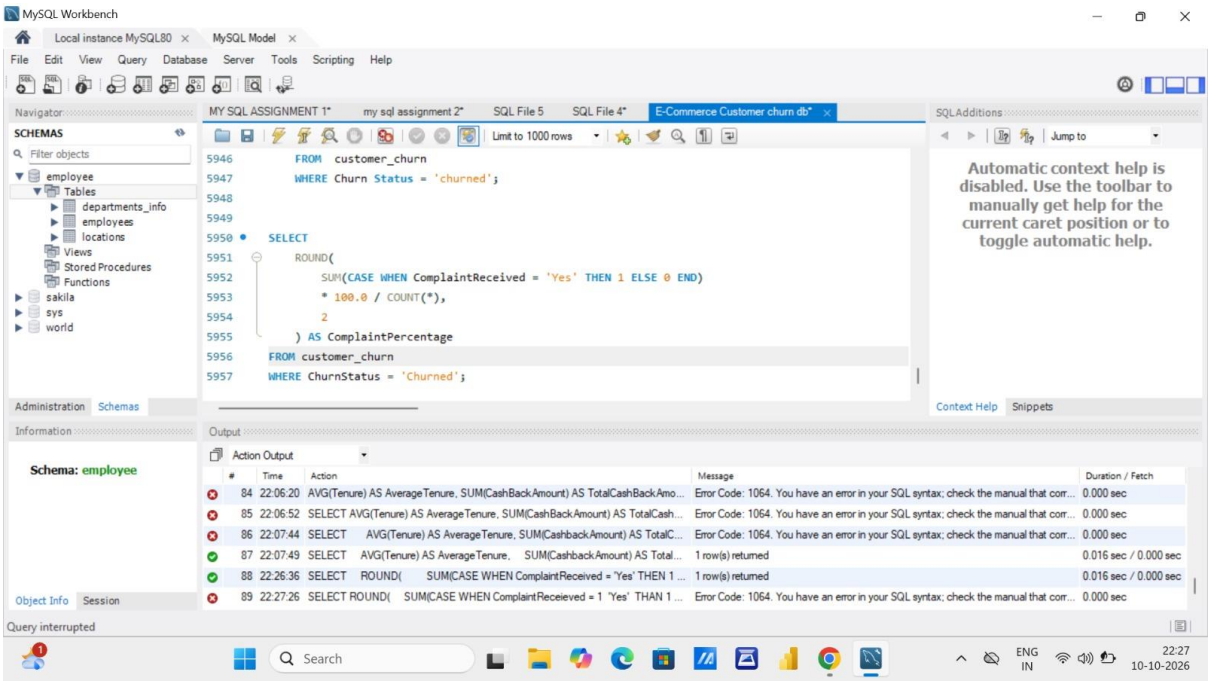
SQL query

```
SELECT ROUND(SUM(CASE WHEN ComplaintReceived = 'Yes' THEN 1 ELSE 0 END)
             * 100.0 / COUNT(*), 2) AS ComplaintPercentage
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Result

ComplaintPercentage
53.59

508 of the 948 churned customers had made a complaint, so complaints are strongly linked to chur



Task 4 - City tier with the most churned Laptop & Accessory customers

Churned customers who prefer Laptop & Accessory are grouped by CityTier, sorted from highest to lowest, and the top row is kept with LIMIT 1.

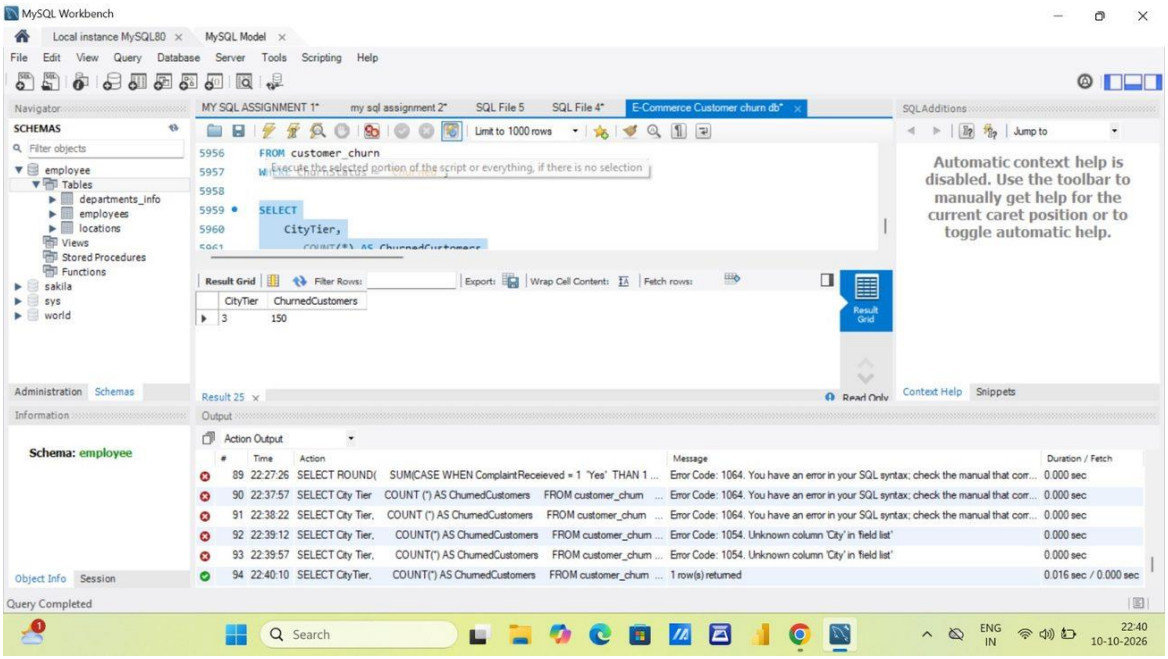
SQL query

```
SELECT CityTier, COUNT(*) AS ChurnedCustomers
FROM customer_churn
WHERE ChurnStatus = 'Churned'
  AND PreferredOrderCat = 'Laptop & Accessory'
GROUP BY CityTier
ORDER BY ChurnedCustomers DESC
LIMIT 1;
```

Result

CityTier	ChurnedCustomers
3	150

City Tier 3 has the most churned Laptop & Accessory customers (150), ahead of Tier 1 (60).



Task 5 - Most preferred payment mode among active customers

Active customers are grouped by payment mode, and the mode with the highest count is kept.

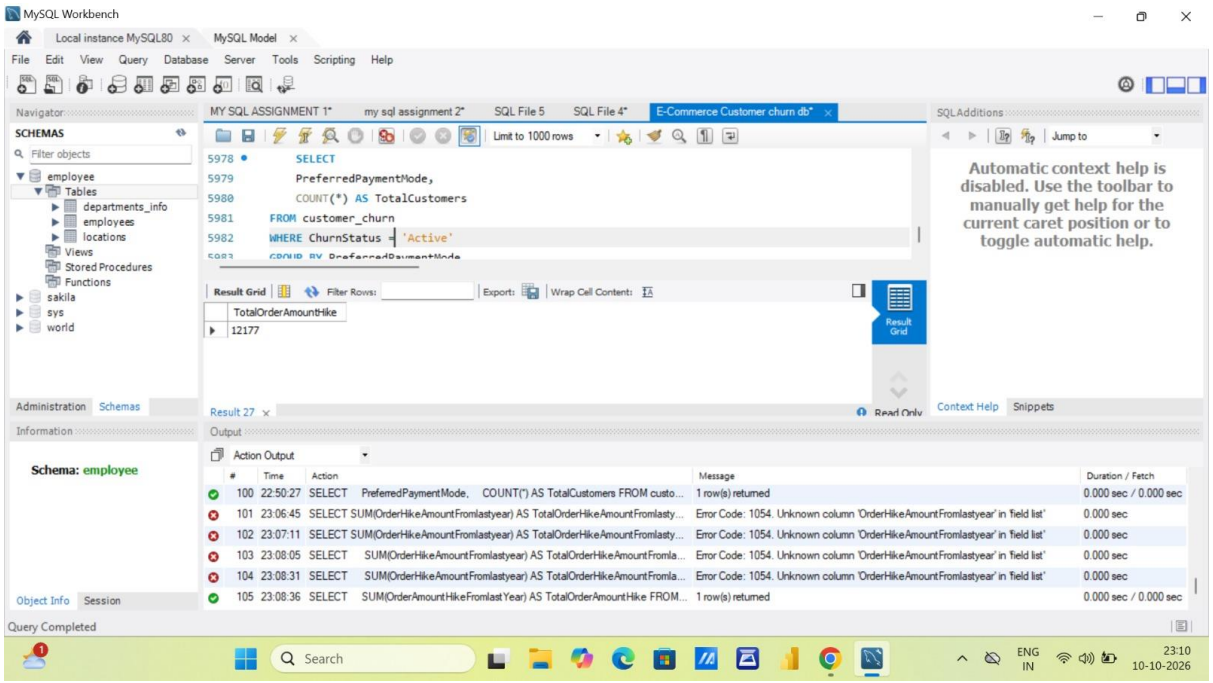
SQL query

```
SELECT PreferredPaymentMode, COUNT(*) AS TotalCustomers
FROM customer_churn
WHERE ChurnStatus = 'Active'
GROUP BY PreferredPaymentMode
ORDER BY TotalCustomers DESC
LIMIT 1;
```

Result

PreferredPaymentMode	TotalCustomers
Debit Card	1956

Debit Card is the most popular payment mode among active customers (1,956), followed by Credit Card (1,522).



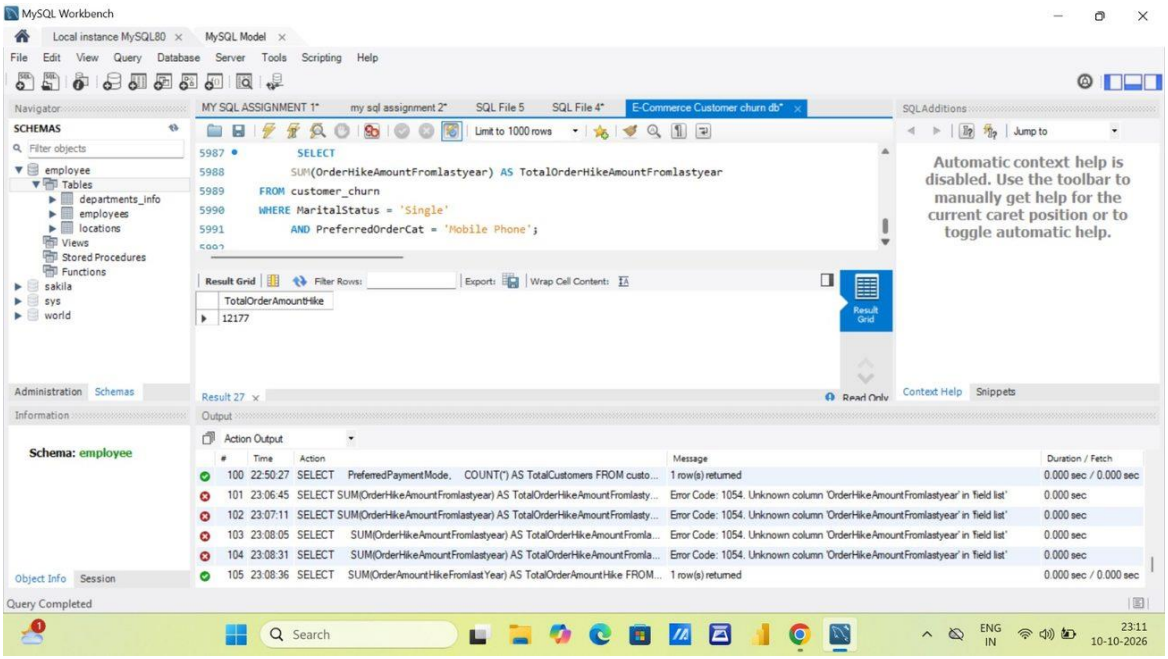
Task 6 - Total order amount hike for single customers who prefer Mobile Phone

The query filters on MaritalStatus = Single and PreferredOrderCat = Mobile Phone, then adds up OrderAmountHikeFromlastYear.

SQL query

```
SELECT SUM(OrderAmountHikeFromlastYear) AS TotalOrderAmountHike
FROM customer_churn
WHERE MaritalStatus = 'Single'
  AND PreferredOrderCat = 'Mobile Phone';
```

Result



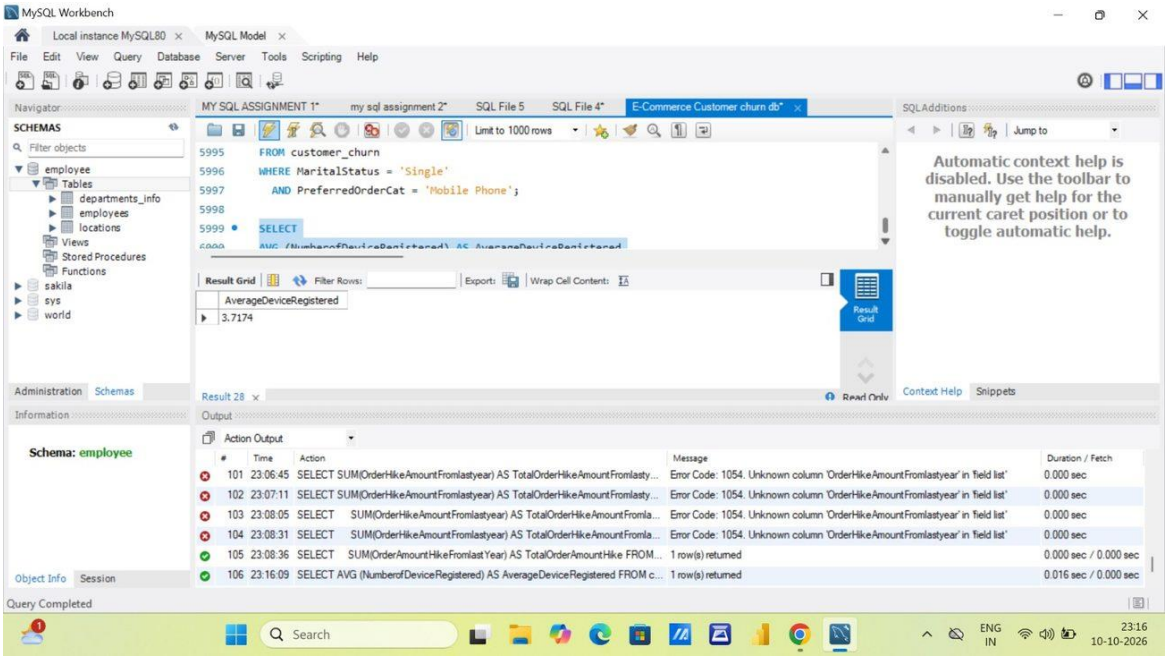
Task 7 - Average devices registered by UPI users

Customers whose payment mode is UPI are selected and AVG is applied to NumberOfDeviceRegistered.

SQL query

```
SELECT AVG(NumberOfDeviceRegistered) AS AverageDeviceRegistered
FROM customer_churn
WHERE PreferredPaymentMode = 'UPI';
```

Result



Task 8 - City tier with the highest number of customers

Customers are grouped by CityTier and counted, and the largest group is returned.

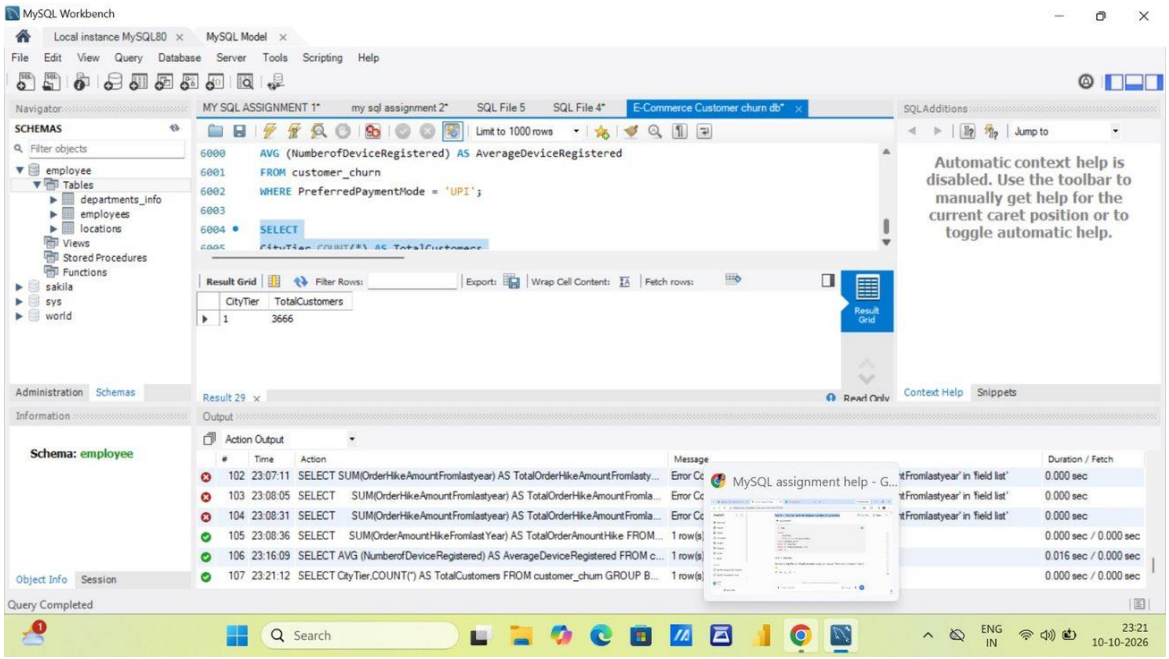
SQL query

```
SELECT CityTier, COUNT(*) AS TotalCustomers
FROM customer_churn
GROUP BY CityTier
ORDER BY TotalCustomers DESC
LIMIT 1;
```

Result

CityTier	TotalCustomers
1	3666

City Tier 1 has the most customers (3,666)



Task 9 - Gender that used the highest number of coupons

The coupons used are added up for each gender with SUM(CouponUsed), and the larger total is shown first.

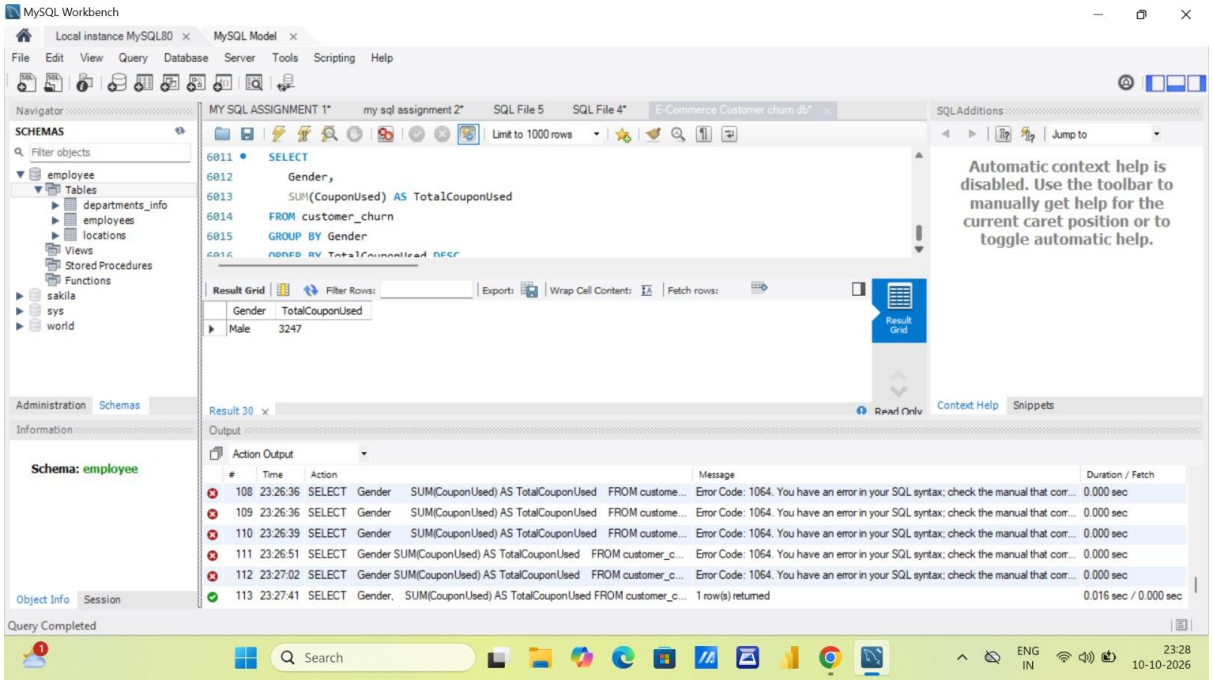
SQL query

```
SELECT Gender, SUM(CouponUsed) AS TotalCouponUsed
FROM customer_churn
GROUP BY Gender
ORDER BY TotalCouponUsed DESC
LIMIT 1;
```

Result

Gender	TotalCouponUsed
Male	5629

Male customers used more coupons (5,629) than female customers (4,034).



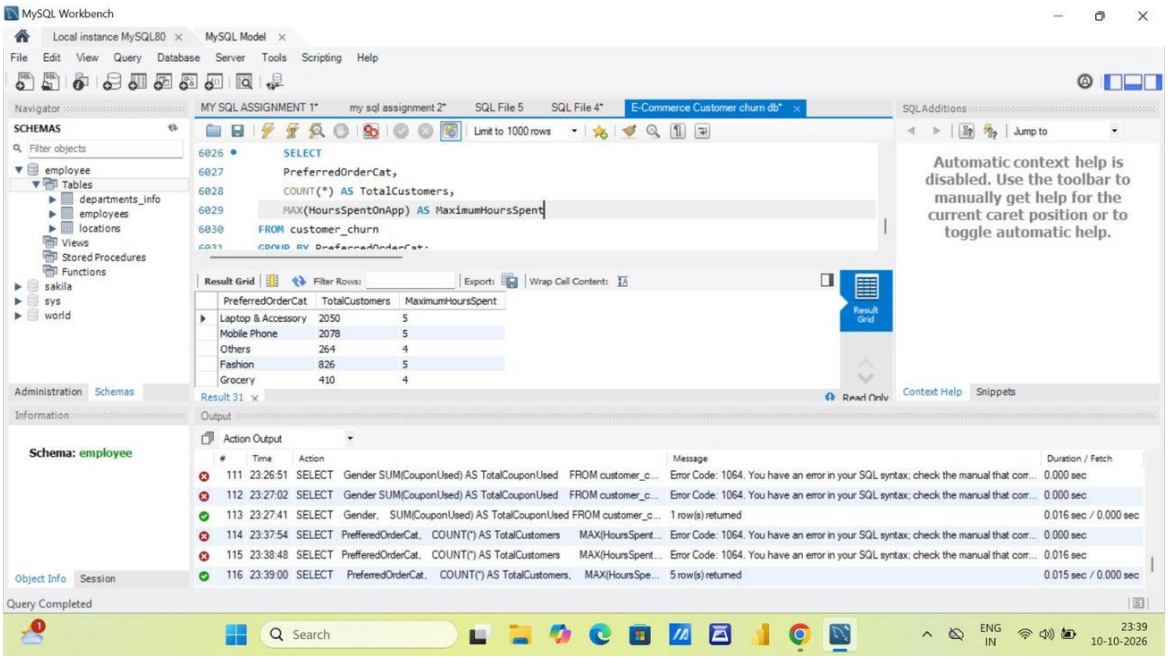
Task 10 - Customers and maximum app hours in each order category

Customers are grouped by PreferredOrderCat. COUNT(*) gives the number of customers and MAX(HoursSpentOnApp) gives the highest value in each group.

SQL query

```
SELECT PreferredOrderCat,
       COUNT(*) AS TotalCustomers,
       MAX(HoursSpentOnApp) AS MaximumHoursSpent
FROM customer_churn
GROUP BY PreferredOrderCat;
```

Result



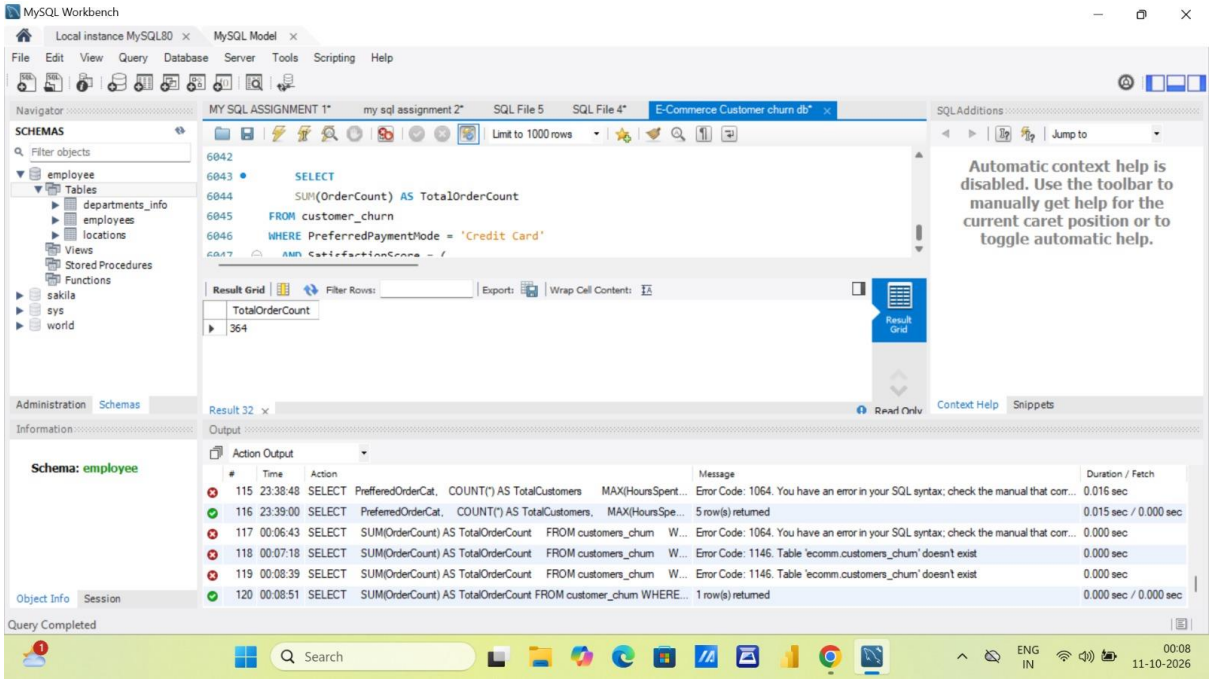
Task 11 - Total orders of credit card users with the maximum satisfaction score

A subquery finds the highest satisfaction score in the table (5). The main query then adds up OrderCount for credit card customers who have that score.

SQL query

```
SELECT SUM(OrderCount) AS TotalOrderCount
FROM customer_churn
WHERE PreferredPaymentMode = 'Credit Card'
  AND SatisfactionScore = (SELECT MAX(SatisfactionScore) FROM customer_churn);
```

Result



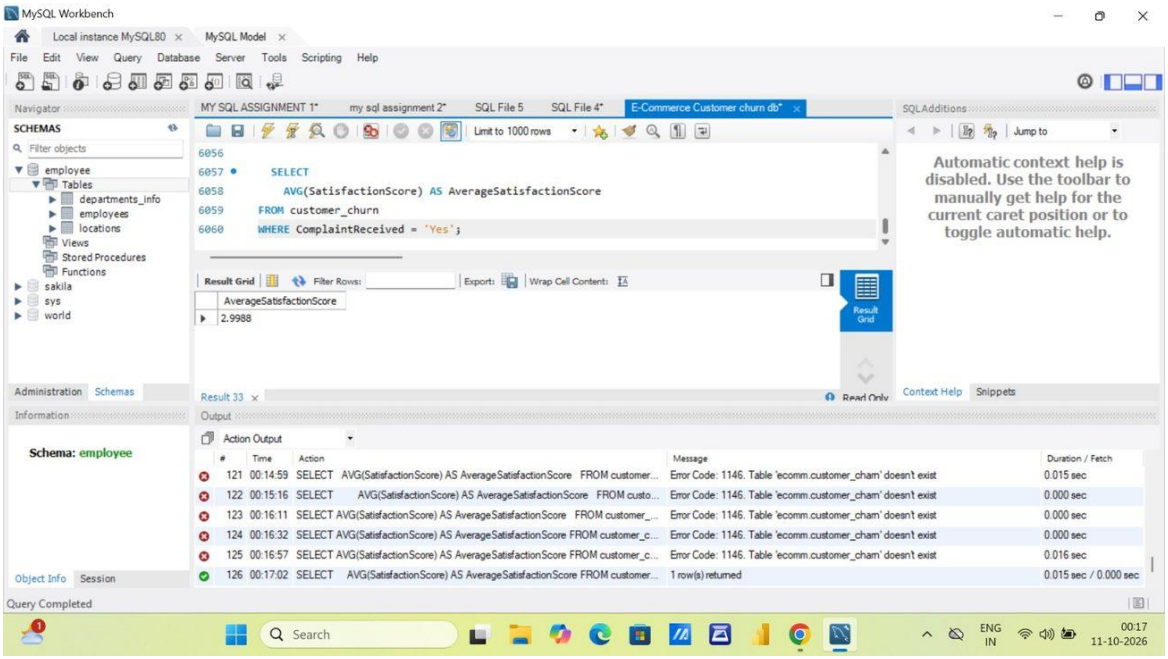
Task 12 - Average satisfaction score of customers who complained

Customers with ComplaintReceived = Yes are selected and AVG is applied to SatisfactionScore.

SQL query

```
SELECT AVG(SatisfactionScore) AS AverageSatisfactionScore
FROM customer_churn
WHERE ComplaintReceived = 'Yes';
```

Result



Task 13 - Preferred order categories of customers who used more than 5 coupons

The WHERE clause keeps customers with CouponUsed > 5. DISTINCT removes duplicates so each category appears once, and ORDER BY sorts them alphabetically.

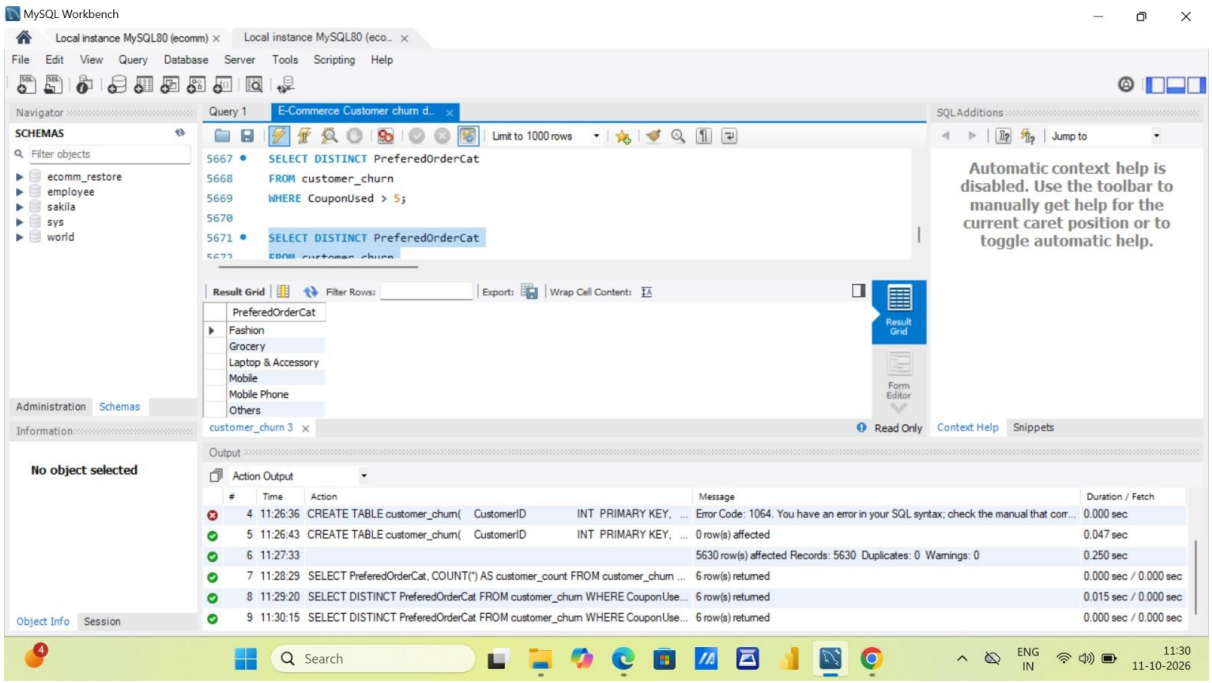
SQL query

```
SELECT DISTINCT PreferredOrderCat
FROM customer_churn
WHERE CouponUsed > 5
ORDER BY PreferredOrderCat;
```

Result

PreferredOrderCat
Fashion
Grocery
Laptop & Accessory
Mobile Phone
Others

303 customers used more than 5 coupons. Laptop & Accessory (99) and Fashion (89) are the largest groups among them.



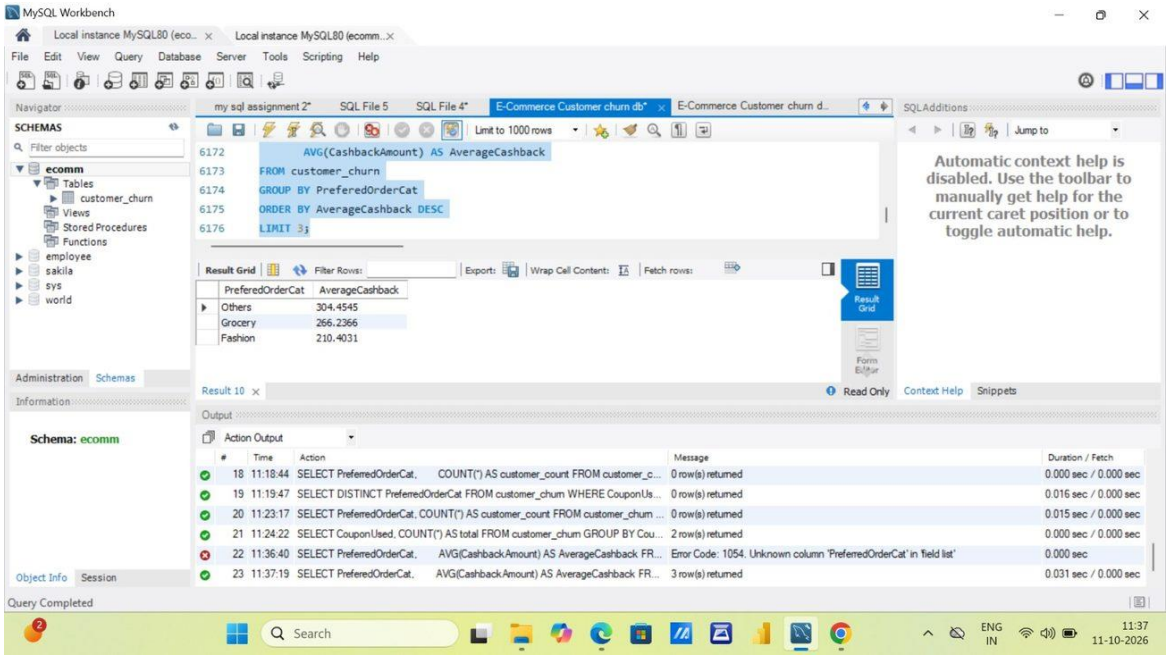
Task 14 - Top 3 order categories by average cashback

The average cashback is calculated for each category, sorted from highest to lowest, and LIMIT 3 keeps the first three.

SQL query

```
SELECT PreferredOrderCat, AVG(CashbackAmount) AS AverageCashback
FROM customer_churn
GROUP BY PreferredOrderCat
ORDER BY AverageCashback DESC
LIMIT 3;
```

Result



Task 15 - Payment modes with average tenure of 10 months and more than 500 orders

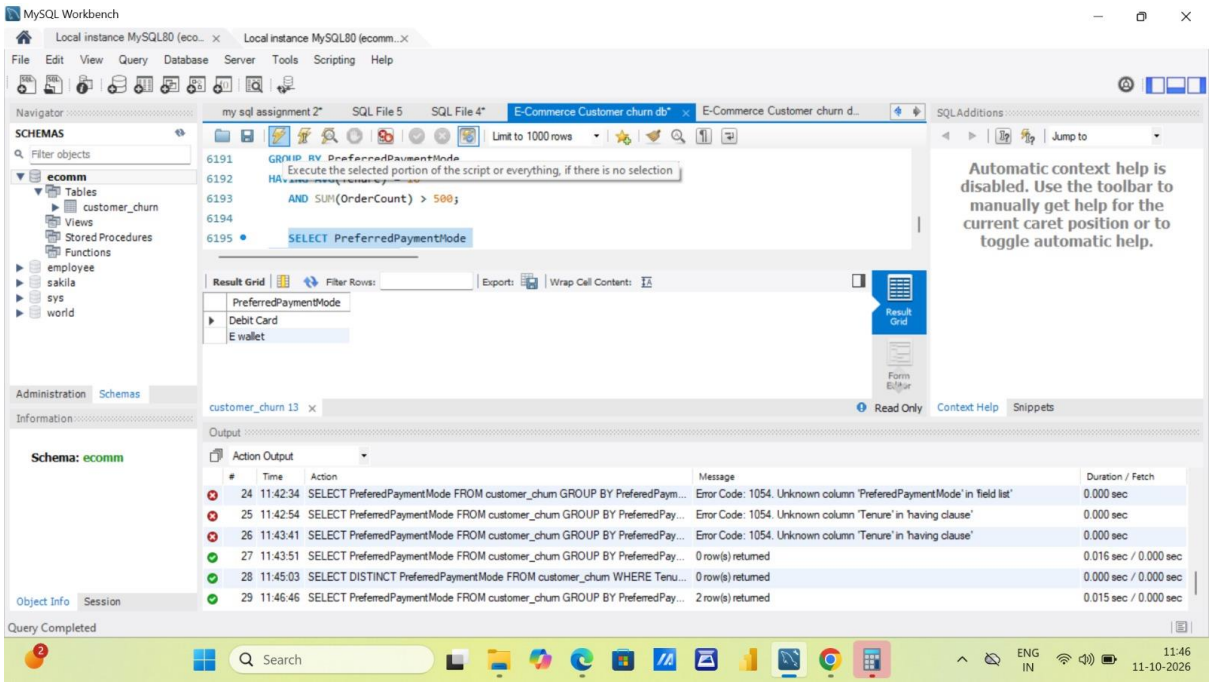
Because the conditions are on group totals, HAVING is used instead of WHERE. The average tenure is rounded to a whole number because no payment mode has an average of exactly 10.00. SUM(OrderCount) > 500 keeps only the modes with more than 500 orders.

SQL query

```
SELECT PreferredPaymentMode
FROM customer_churn
GROUP BY PreferredPaymentMode
HAVING ROUND(AVG(Tenure)) = 10
   AND SUM(OrderCount) > 500;
```

Result

PreferredPaymentMode	Average tenure	Total orders
Credit Card	9.78	5409
Debit Card	9.95	6802
E wallet	10.20	1849



Task 16 - Churn breakdown by distance category

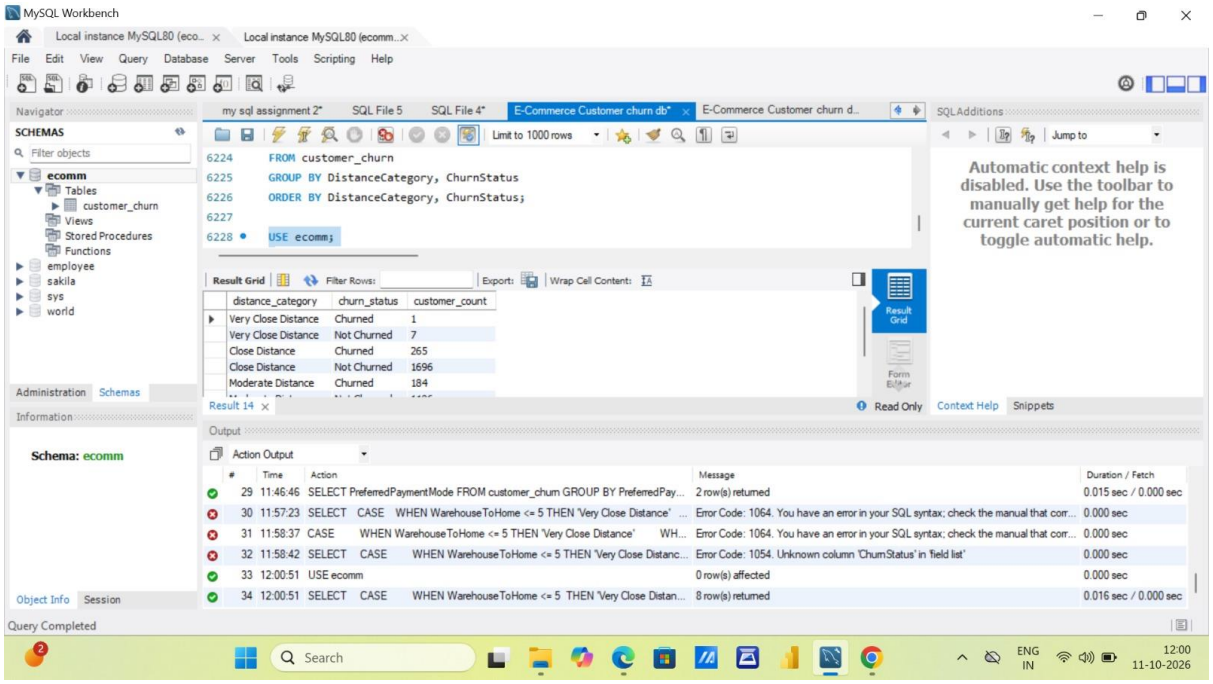
A CASE statement turns the numeric distance into four labels. Customers are then grouped by distance category and ChurnStatus and counted. FIELD() in the ORDER BY lists the categories from nearest to farthest.

SQL query

```
SELECT CASE
           WHEN WarehouseToHome <= 5  THEN 'Very Close Distance'
           WHEN WarehouseToHome <= 10 THEN 'Close Distance'
           WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
           ELSE 'Far Distance'
       END AS DistanceCategory,
       ChurnStatus,
       COUNT(*) AS CustomerCount
FROM customer_churn
GROUP BY DistanceCategory, ChurnStatus
ORDER BY FIELD(DistanceCategory, 'Very Close Distance', 'Close Distance',
               'Moderate Distance', 'Far Distance'), ChurnStatus;
```

Result

DistanceCategory	ChurnStatus	CustomerCount
Very Close Distance	Active	7
Very Close Distance	Churned	1
Close Distance	Active	1696
Close Distance	Churned	265
Moderate Distance	Active	1106
Moderate Distance	Churned	184
Far Distance	Active	1871
Far Distance	Churned	498



Task 17 - Married City Tier 1 customers with above-average orders

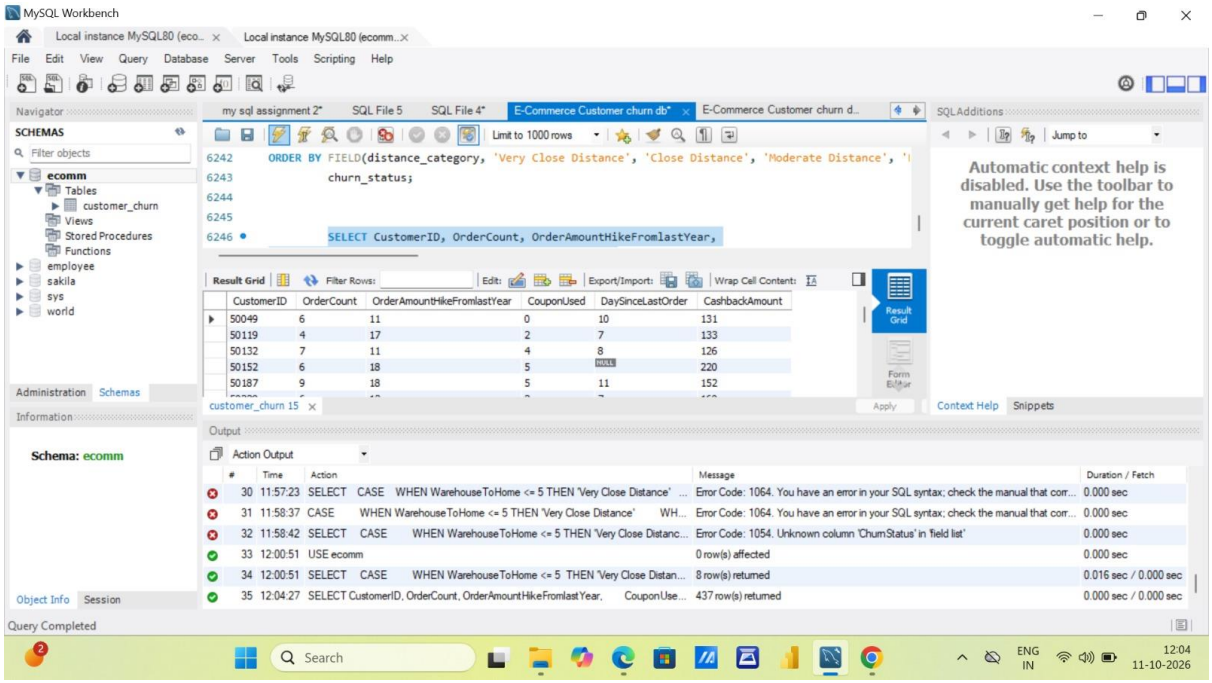
A subquery calculates the average OrderCount of all customers . The main query keeps married customers in City Tier 1 whose OrderCount is higher than that average, which means 3 or more orders.

SQL query

```
SELECT CustomerID, OrderCount, OrderAmountHikeFromlastYear,
       CouponUsed, DaySinceLastOrder, CashbackAmount
FROM customer_churn
WHERE MaritalStatus = 'Married'
  AND CityTier = 1
  AND OrderCount > (SELECT AVG(OrderCount) FROM customer_churn);
```

Result

The query returns 576 customers on the cleaned table. Each row shows that customer's order details.



Task 18 - Customer returns

a) Create the customer_returns table and insert the data

Explanation: A new table is created in the ecomm database with ReturnID as the primary key. The eight return records from the assignment are then inserted with one INSERT statement, and a SELECT confirms they were stored.

```
CREATE TABLE customer_returns (
    ReturnID     INT PRIMARY KEY,
    CustomerID   INT,
    ReturnDate   DATE,
    RefundAmount INT
);

INSERT INTO customer_returns (ReturnID, CustomerID, ReturnDate, RefundAmount) VALUES
    (1001, 50022, '2023-01-01', 2130),
    (1002, 50316, '2023-01-23', 2000),
    (1003, 51099, '2023-02-14', 2290),
    (1004, 52321, '2023-03-08', 2510),
    (1005, 52928, '2023-03-20', 3000),
    (1006, 53749, '2023-04-17', 1740),
    (1007, 54206, '2023-04-21', 3250),
    (1008, 54838, '2023-04-30', 1990);

SELECT * FROM customer_returns;
```

Result

The SELECT returns all 8 rows, which confirms the data was inserted correctly.

b) Return details of customers who churned and complained

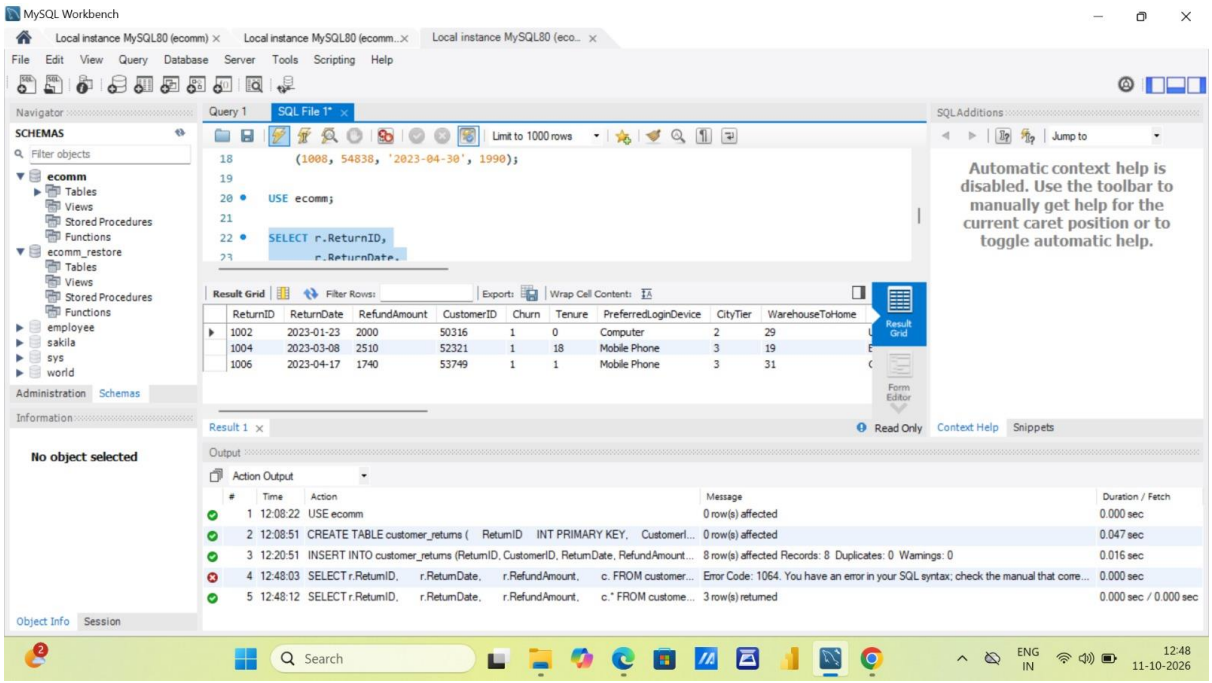
Explanation: An INNER JOIN links each return to its customer through CustomerID. The WHERE clause then keeps only customers with ChurnStatus = Churned and ComplaintReceived = Yes, so the output shows the return details together with the customer details.

```
SELECT r.ReturnID, r.ReturnDate, r.RefundAmount, c.*
FROM customer_returns r
JOIN customer_churn c
  ON r.CustomerID = c.CustomerID
WHERE c.ChurnStatus = 'Churned'
  AND c.ComplaintReceived = 'Yes';
```

Result

ReturnID	CustomerID	ReturnDate	RefundAmount
1002	50316	2023-01-23	2000
1004	52321	2023-03-08	2510
1006	53749	2023-04-17	1740

Three of the eight returns belong to customers who both complained and churned. The other five are excluded because the customer did not churn, did not complain, or both. Each row also shows all the customer columns (tenure, device, gender and so on).



7. Conclusion

This assignment used MySQL to clean, transform and analyse an e-commerce customer dataset. The data was first cleaned by filling missing values (mean for continuous columns and mode for count columns), removing two unrealistic distance outliers and merging inconsistent labels. It was then transformed with clearer column names and readable ChurnStatus and ComplaintReceived columns. The analysis answered 18 questions using filtering, grouping, aggregate functions, subqueries, CASE statements, HAVING and a join.

The results show that churn is concentrated among newer customers, customers who complained and customers who live far from the warehouse. Complaints stand out most, because more than half of the customers who left had raised one. Payment and category patterns also give useful targets, such as the large Debit Card and Credit Card customer base and the high churn of Laptop & Accessory customers in City Tier 3.